What's new

QED version 2.0: an interactive text in first-order logic

Terence Tao · 18 August 2018

https://terrytao.wordpress.com/2018/08/18/qed-version-2-0-an-interactive-text-in-first-order-logic/

tao2typ · converted to Typst 2026-10-14 (UTC)

As readers who have followed my previous post will know, I have been spending the last few weeks extending my previous interactive text on propositional logic (entitied "QED") to also cover first-order logic. The text has now reached what seems to be a stable form, with a complete set of deductive rules for first-order logic with equality, and no major bugs as far as I can tell (apart from one weird visual bug I can't eradicate, in that some graphics elements can occasionally temporarily disappear when one clicks on an item). So it will likely not change much going forward.

I feel though that there could be more that could be done with this sort of framework (e.g., improved GUI, modification to other logics, developing the ability to write one's own texts and libraries, exploring mathematical theories such as Peano arithmetic, etc.). But writing this text (particularly the first-order logic sections) has brought me close to the limit of my programming ability, as the number of bugs introduced with each new feature implemented has begun to grow at an alarming rate. I would like to repackage the code so that it can be re-used by more adept programmers for further possible applications, though I have never done something like this before and would appreciate advice on how to do so. The code is already available under a Creative Commons licence, but I am not sure how readable and modifiable it will be to others currently. [*Update*: it is now on GitHub.]

[One thing I noticed is that I would probably have to make more of a decoupling between the GUI elements, the underlying logical elements, and the interactive text. For instance, at some point I made the decision (convenient at the time) to use some GUI elements to store some of the state variables of the text, e.g. the exercise buttons are currently storing the status of what exercises are unlocked or not. This is presumably not an example of good programming practice, though it would be relatively easy to fix. More seriously, due to my inability to come up with a good general-purpose matching algorithm (or even specification of such an algorithm) for the the laws of first-order logic, many of the laws have to be hard-coded into the matching routine, so one cannot currently remove them from the text. It may well be that the best thing to do in fact is to rework the entire codebase from scratch using more professional software design methods.]

[*Update*, Aug 23: links moved to GitHub version.]

Comments (64)

Francisco Javier Thayer · 18 August 2018

There are already a siizeable number of “Theorem provers” (or “proof checkers). Among these are HOL the Mizar system and Coq (which formalized a proof of the Thomson Feit Theorem). The field has been chugging along for about 40 years with many more systems, too many to be included here (including one which I was heavily involved in not mentioned.) The holy grail of the field is to come up w/ a system that can actually FIND proofs, and some do make considerable headway in that direction, such as HOL.

> **Anonymous** · 20 August 2018
>
> This is a game that guides you to discover theorems, developing the subject from scratch as you go. It’s not just a proof checker.

Nisarg Kothari · 19 August 2018

I suggest you post the code to github.com, which is the de facto standard for open source code sharing and collaboration.

cjerdonek · 19 August 2018

Congrats on your efforts! As a software developer, the first thing I’d suggest is choosing an open source license rather than a CC license, as open source licenses are specifically designed for (and more appropriate for) code. You can’t really go wrong with something simple like MIT or BSD-3-Clause, unless you want a copyleft license like GPLv3. You can find a list with instructions here: https://opensource.org/licenses/category
And yes, I’d echo Nisarg’s comment to put your code on GitHub as the next step after that if you’d like to solicit help from other software engineers. That by itself will make it much easier for others to view, download, and offer you specific suggestions, etc.

cjerdonek · 19 August 2018

PS – you also want to be sure to include a copyright notice along with any license (as the instructions for adding a license should tell you). While it might seem pedantic, it’s good to do from the beginning so that subsequent sharing and collaboration can be on clear footing.
GitHub has good documentation for creating your first repository (including things like adding a README, which serves as the “title page” for your project, as well as adding a license file, etc):
https://help.github.com/articles/creating-a-repository-on-github/

dr anil pedgaonkar · 19 August 2018

congrats a great work! dr anil pedgaonkar
i would not accept zf system.
but would accept empty sewt and sets constructed from it using ZF axioms except the ac and the last axiom in book by marseden.
we define a swt is only the one which is constructed as above . so no abstract sets.. i would be happy if great terrance tao replies to my comment. mail anilped@hotmail.com

First order logic – The nth Root · 19 August 2018

[…] As readers who have followed my previous post will know, I have been spending the last few weeks extending my previous interactive text on propositional logic (entitled “QED”) to also cover first-order logic. The text has now reached what seems to be a stable form, with a complete set of deductive rules for first-order logic with equality, and no major bugs as far as I can tell (apart from one weird visual bug I can’t eradicate, in that some graphics elements can occasionally temporarily

disappear when one clicks on an item). So it will likely not change much going forward. ... (Terence Tao) [...]

 **LSAT** · 20 August 2018

This is a great little app, but I would suggest some sort of help function or way to bypass questions if needed. No matter what I do, I cannot make 6.1(b) work. I think it's more of a quirk with the app than my logic, but in either case I'm doing something that just won't work.

 **Terence Tao** · 20 August 2018

By "not work", do you mean that you were expecting the text to perform in a certain way and it wasn't (i.e. a potential bug in the code), or that you weren't able to work out the solution? In the latter case, the discussion provided below Exercise 6.1(b) indicates the general strategy one needs to solve this sort of problem (in which the desired conclusion is of the form "X IMPLIES Y".
Also, solving Exercise 6.1(b) is not required in order to unlock later exercises (only the first exercise of a section needs to be solved in order to unlock the rest of the section, as well as the first exercise of the next section).

 **LSAT** · 20 August 2018

Hello Dr. Tao, what I mean is that I find the instructions unclear. I can create the formula for A AND (A OR B), but everything I try after doesn't seem to work. I note a comment above about Macs, which I use. I haven't tried a browser other than Chrome, but I think it's not the browser. I must simply not be using the app correctly.
I'd like to see the actual procedure (click this, drag that) for the problem.

 **Terence Tao** · 20 August 2018

If you have reached as far as 6.1(b), the law of implication introduction should be unlocked, so that when one clicks on the formula A AND (A OR B), the ability to deduce A AND (A OR B) (assuming A AND (A OR B)) should be available.
Sometimes with version updates, laws that were previously unlocked become locked again. This can usually be fixed by revisiting the exercise that unlocked the law (in this case, 4.1). The achievements window will tell you what laws are currently available. If all else fails one can reset the text.

 **Jacob H.** · 20 August 2018

Hi Terry! You might consider adding event.metaKey to the event.ctrlKey-catching clickBox function in gui.js so that mac users can also multi-select items (change the line "if (event.ctrlKey)" to "if (event.ctrlKey || event.metaKey)")
[Now implemented; thanks for the suggestion -T.]

 **Siddhartha Gadgil** · 20 August 2018

This is really nice and useful. I would be happy to contribute.
Something I have done on a much smaller scale is embedded illustration such as http://math.iisc.ac.in/~gadgil/MA261/notes/chapter-3.html#percolation" in notes of Manjunath Krishnapur (converted from LaTeX), also in standalone form such as http://math.iisc.ac.in/~gadgil/MA261/illustrations/birthdays.html to be used in lectures.
I too would echo the suggestion to put the code on Github as suggested (or Gitlab, which has some advantages, including latex support)? It makes it easy not only to contribute, but to plan using issues etc.

The two quick changes I would suggest are delegating styling in part to something like Bootstrap (which also makes the page mobile friendly) and delegating updating to a framework such as Vue. I would be happy to contribute these.

 **Pedro Sánchez Terraf** · 21 August 2018

Dear Terence,
Thank you very much for your development. I've tried the app up to
12.1(a) ("Reductio ad absurdum") Given B AND (NOT B) [assuming A]: deduce NOT A.
There is a detail to that exercise. Any propositional theorem of the form "NOT A" (no matter the assumption set) is provable without Excluded Middle (this is Glivenko theorem, and it doesn't go through for predicate logic: http://p.sanchezterraf.com.ar/2015/10/22/how-about-a-little-absurdity/).
This applies to the system of "natural deduction" of Gentzen and similar ones.
Best wishes.

 **Rolfe Schmidt** · 21 August 2018

I agree that a lot can be done with this, and it would be great to turn it into something that was easy for others to augment and use. I agree with the earlier commenters that a great way to start this is to put the code in a public repository like GitHub. This would allow us to have focused technical discussions and contribute new code in a controlled way.
You are spot on about the need to decouple the presentation from the "game data" from the logic engine. There are many ways to do this, of course, and design decisions need to be based on the expected users. On the spot, I'd start by moving the GUI to a framework like ReactJS and move the interactive text and other "game data" into a JSON configuration file so that it is ajax ready.
Those might not be very interesting tasks, but they are things that some of us could hammer out quickly once it's on github. In any case, I'd be happy to lend a hand here.

 **daniel peters** · 21 August 2018

adding to the voices to put the source of the site on github, many many people (myself included) would love to be able to use and contribute to a text like this.

 **Terence Tao** · 21 August 2018

OK, I think I managed to create a github repository for the code here: https://github.com/teorth/QED
I assume now that people are free to clone it to work on it separately? I'm still not completely comfortable with the git-style version control paradigm.

> **cjerdonek** · 21 August 2018
>
> Good work! Yes, Git's command usage is a bit unintuitive and takes some getting used to, though the core mental model of the commit history as a directed acyclic graph is somewhat elegant.
> A couple more things to consider: 1) You may want to use the "issues" tracker as a way to coordinate and ask for help on things: https://github.com/teorth/QED/issues
> 2) It will probably be good to have an automated or semi-automated way of publishing to the web the latest version of what's in the repository. One option for doing this would be to use "GitHub Pages" (the "Project Page" variant):
> https://help.github.com/articles/configuring-a-publishing-source-for-github-pages/
> This will wind up publishing the site to the following URL (if published from your version):
> https://teorth.github.io/QED
> I expect that people will either suggest changes to you that you can accept through GitHub's UI, or else someone might take the lead on developing improvements in their own fork. It partly depends on whether you want to be the one to manage the main version. (You will also have

the option of granting one or more trusted individuals “write” access to your own version of the repository, so that your version can remain the main one, with others able to work on it without your involvement.)



Siddhartha Gadgil · 21 August 2018

The rest of us will fork (make our copies), clone these and generate pull requests. I just generated one to move files to base.
The automatically generated page for my version (with no changes so far) is at https://siddhartha-gadgil.github.io/QED/QED.html

cjerdonek · 21 August 2018

> But writing this text (particularly the first-order logic sections) has brought me close to the limit of my programming ability, as the number of bugs introduced with each new feature implemented has begun to grow at an alarming rate.
By the way, the main way a software engineer handles this situation is by writing tests. Ideally, much of the code should be structured as testable functions, and a single test would consist of (1) the input to one of these functions, and (2) the expected output. Tests can be set up so they are all run by invoking a command and even e.g. after pushing to GitHub. In that way you can be sure you’re not breaking previous code when adding new code, etc. It takes a bit of work to set that up, but that’s what you should be aiming for to have a more maintainable code base.



nicolaspatrois · 22 August 2018

Hi,
I suspect a bug in exercise 6.1(a), it seems that I can’t use the Deduction theorem and I don’t know why.
Should I prove again 5.1 or 5.2?



Terence Tao · 22 August 2018

Hmm, I can’t replicate this issue. Is the deduction theorem unlocked in the achievements window? If you drag A (assuming A) into the root environment, does it say that no available deductions can be formed?



nicolaspatrois · 22 August 2018

The Deduction theorem is unlocked.
I see this:
Available deductions
From formula “A”, [root environment]:
IMPLICATION INTRODUCTION: A [assuming A]
EXERCISE 4.1: A AND A [assuming A]
But when I click on the first, I can’t use the Deduction theorem when A is in the root environment even if I built the formula A IMPLIES.



Terence Tao · 22 August 2018

To use the deduction theorem, you have to start with a statement that is in an assumption environment, and drag it out of that environment. Dragging a formula from the Formulas window won’t trigger the deduction theorem.
To solve this particular exercise, first use the implication introduction law to derive A assuming A, then you can drag A from the “assuming A” environment out to the root environment to trigger the deduction theorem.

 **nicolaspatrois** · 22 August 2018

OK, thank you.

 **Yahya Abdal-Aziz** · 22 August 2018

Terry, in ex. 3.1(a), where you introduce formulae, the text reads in part “The formula A is known to be true, …”. However, what we know to be true is the statement A. Is there perhaps a missing rule of deduction that allows us to assert that if statement A is true, then formula “A” is true?

 **Terence Tao** · 22 August 2018

I’ve now reworded the text to hopefully make things more clear. The sentence A is available as a formula (and hence can be placed in the Formulas window as a formula, but is also known to be true in this exercise, and so can also be placed in the Root Environment as a statement. Basically, the root environment is the location in the text for holding all the sentences that one is sure is true, while the formulas window is an additional location that can be used to hold sentences that may or may not be true. So certainly every statement in the root environment can also become a formula. (Indeed, the text allows you to drag sentences from an environment to the formulas window, which is sometimes a faster way of making formulas than using the formula formation rules.)

 **Yahya Abdal-Aziz** · 23 August 2018

Thanks! That’s certainly clearer. BTW, minor typo on screen: “Root enviroment” should read “Root environment”.
[Corrected, thanks – T.]

 **P** · 22 August 2018

Are you planning to add more exercises? The amount of exercises in the late half are kinda lacking compared to the rest. This should only involve adding things to the main html only, with almost no change to the logic code.

 **Terence Tao** · 23 August 2018

Fair enough. I have added two more exercises to Section 23 (Russell’s paradox, and no largest natural number) and one more exercise to Section 22 (a Lewis Carroll logic puzzle) to the most current version of the text, now hosted on Github at https://teorth.github.io/QED/QED.html . If more good exercises come to mind I will add them also (and people are now welcome to suggest their own exercises, or even contribute them directly to the git repository).

 **Terence Tao** · 24 August 2018

Some more exercises added. Three of them are in group theory: the cancellation law (XY = XZ implies Y=Z), the fact that the product of two invertible elements in a monoid is still invertible, and the fact that group inversion is an involution. A further (particularly challenging) exercise has been added: to prove that it is possible to raise an irrational positive real to an irrational positive real and obtain a rational. (This is a classic application of the law of the excluded middle; quite fun to work out, if one hasn’t seen it before.) Also added a couple of short helper exercises that shorten the proofs of some of the lengthier exercises.

 **Anonymous** · 25 August 2018

Dear Terence, thanks being awesome enough to dedicate your time for such projects, it’s truly generous of your part! :)

One trivial enhancement: Please add a warning when JS is disabled since the game requires it to work. It's pretty easy to do with the HTML tag https://www.w3schools.com/TAGs/tag_noscript.asp
[Done – T.]

cjerdonek · 26 August 2018

Question: can you point out generally where in the code this comment is a reference to? "More seriously, due to my inability to come up with a good general-purpose matching algorithm (or even specification of such an algorithm) for the the laws of first-order logic, many of the laws have to be hard-coded into the matching routine, so one cannot currently remove them from the text."
Maybe I can think a bit about possible alternate approaches.

> **Terence Tao** · 26 August 2018
>
> The hardcoded laws are universalIntroduction, universalIntroduction2, universalSpecification, universalSpecification2, existentialInstantiation, existentialInstantiation2, and indiscernability. They've now been moved to logic.js, so the issue of requiring the main HTML (or js) files to contain these laws is no longer an issue. However there is a significant amount of code required to match these specific laws (lines 770-1092 of logic.js), compared against the general-purpose matching algorithm (lines 1195-1417).
> There are two main issues. The first is that if one wishes to apply a law that involves an expression such as $P(\alpha)$, then there can be multiple ways to match the expression one has to give multiple choices for P and α. For instance, if one's expression is $Q(x,x)$ and one wishes to match this with $P(\alpha)$, then one could take P to be the predicate $P(y) = Q(y,y)$ and α to be x, or P could be the predicate $P(y) = Q(y,x)$ and $\alpha = x$, or $P(y) = Q(x,y)$ and $\alpha = x$, or $P(y) = Q(x,x)$ and α is an arbitrary term. The latter is particularly annoying since it means that there are actually an infinite number of matching possibilities, though of course there are only finitely many inequivalent ones. Among other things, it means that some laws (specifically, indiscernability and the existential introduction laws) give rise to multiple matches from a given set of inputs (in the final exercise, for instance, there is a point in the proof where existential introduction makes $2^6 = 64$ matches!). The general-purpose matching algorithm I have presumes that the matching is unique, and so cannot handle this sort of multiplicity; in the two laws where multiplicity does occur, I had to use ad hoc code to create the matches.
> The second issue, which I view as the more serious one, is that some laws come with side conditions, e.g. a certain expression in the law is not allowed to use bound variables, or any free variables not currently available in the target environment. I haven't thought of a good way to encode these side conditions in general, and they are another reason why the hard-coded laws need separate code to treat.
> Incidentally neither of these issues show up in the propositional logic portion of the text (Sections 1-13). For that portion any law is treatable by the general-purpose matching algorithm.

Antoine Deleforge · 27 August 2018

I noted some typos in the last exercises:
-24.1(a) : In the second bullet point, alpha=beta should be alpha=alpha
-24.4 : In the first bullet point, the letter alpha is wrongly displayed as "&alpha".
-24.6 : In the second bullet point, X*1*X should be X*1 = X
[Corrected, thanks – T.]

Lior Silberman · 27 August 2018

(v.2.5.4) how does one use ex.8.2?

I have two statements of the form “A implies B”, “B implies C” where A,B,C are formulas, and want to extract “A implies C” but dragging one onto the other or selecting both doesn’t give list ex.8.2 as one of the options — I only get “conjunction introduction” and “exercise 1.1”.



Lior Silberman · 27 August 2018

Is the problem that 8.2 is phrased in terms of variables rather than in terms of formulas?



Terence Tao · 27 August 2018

I had to change the name of the law in Exercise 8.2 due to an HTML formatting error, with the result being that the law would have become locked again if it were previously unlocked. If one re-does Ex8.2 then the law should become available.
(I recently changed the code to index laws by a shorter, non-HTML name rather than the HTML name, so this issue should eventually no longer become an issue, but legacy players will still have their laws indexed by the HTML name.)



Anonymous · 28 August 2018

This looks awesome!
Dr. Terence, what work is needed for localization? It would be great to have this game translated to other languages and I’d be happy to help on that side but my question is about the technical side. (of course one can fork QED and translate everything, but having everything on the official QED page with a button to change the language would be awesome)



Terence Tao · 28 August 2018

Hmm, I’m not sure technically what the best way to do this is; I’ve opened an issue about this on the github repository at https://github.com/teorth/QED/issues/9

Marc Young · 6 September 2018

Hi Terence,
Thank you for the awesome site! I’m having a lot of fun with it, but I’m currently stuck on Existence (21.1) and I can’t, for the life of me, figure out what’s required. Any hints or suggestions?



Terence Tao · 6 September 2018

You need to use the new Existence axiom to set up an environment where a variable x has been set. Figure out how to establish A inside this environment, then one can pull it back out to the root environment.



Martin Epstein · 7 September 2018

This is fantastic. I hope there is a mobile version some day.
Typo in 18.1, last bullet point. “forn”
[Corrected, thanks – T.]



Martin Epstein · 13 September 2018

18.5 should say “DISTRIBUTES OVER” and not “COMMUTES WITH” I think.
[Changed – T.]

Hugo · 8 September 2018

Could someone give me a suggestion in Exercise 6.1? I can not complete it.
I thought as follows:

1. Assume A;
2. Deduce A implies A (Implication Introduction of 1) – But it does not work.

or
1. Assume A;
2. Deduce A and A (Assuming A);
3. Deduce A (Conjugation Elimination-Assuming A);
4. Deduce A implies A (Implication Introduction of 1 and 3-Assuming A) – But it does not work.



Terence Tao · 8 September 2018

(a) Is "DEDUCTION THEOREM" listed as unlocked in the Achievements section?
(b) to use the deduction theorem, you have to drag a statement inside an assumption environment outside of that environment. For instance, in your first approach, you would take the statement A inside the "Assume A" window and drag it to the root environment. If the deduction theorem is unlocked, A IMPLIES A should be listed as an available deduction.

Hugo · 8 September 2018

Thank you! It worked.
My congratulations for the design of this excellent didactic resource.



Martin Epstein · 20 September 2018

I beat the game the other day. My favorite exercise was Russell's paradox. I was very surprised that we could demonstrate it with basically no assumptions about what the set inclusion operator means. Fun idea for a group theory exercise: Given the equality relation and a binary operation with the following; associativity, a left ID '1', and left invertibility; show that 1 is also a right ID. I was able to prove this in the 24.4 environment in 39 lines.



Martin Epstein · 20 September 2018

38 lines if we ignore the 4th given line which wasn't relevant.



martinepstein · 26 September 2018

I have it down to 30 lines (ignoring the 4th given line). I think the core of the proof is quite elegant, but you need to use associativity on a product of 4 terms which is cumbersome.



Terence Tao · 2 October 2018

I've now implemented this exercise as Exercise 24.8. I was able to establish it in 32 lines (with a slightly different setup), but presumably this can be improved slightly.



Anonymous · 23 September 2018

Is it possible to extend this proof verification software to include also the method based on the univalent foundation of mathematics? see e.g.
https://doi.org/10.1090/bull/1616

William Grosso (@wgrosso) · 24 October 2018

"I would like to repackage the code so that it can be re-used by more adept programmers for further possible applications, though I have never done something like this before and would appreciate advice on how to do so. "
Is that the extent of your goals? Do you have specific applications in mind ("Stochastic Partial Differential Equations: The Game")? How much would you want to be involved? What would be, in your opinion, a very good outcome here?
Open source projects that "succeed" usually have some sort of governance and community structure clearly articulated. Putting it on github and putting an MIT license is a start, but if you want to push it forward more you probably need a vision statement (what's the goal? why should someone

volunteer?) and some sort of idea of how you'd like it to proceed (do you want to be a benevolent dictator? https://en.wikipedia.org/wiki/Benevolent_dictator_for_life? One of 7 people who vote on major changes? Just a guy, playing with code? Etcetera).

 **nicolaspatrois** · 26 October 2018

Hi,
In exercise 9.5(d), I´d like to use 9.1(a) (OR is commutative) but it´s not in the list of available deductions from formulas "A" and "C".
I can deduce that AND is commutative but not OR.
Thank you.

>  **nicolaspatrois** · 26 October 2018
>
> I reproved 9.1(a) but I still did not get the fact that OR is commutative.
>
> >  **Terence Tao** · 27 October 2018
> >
> > To use the commutativity of OR, you have to click on a sentence of the form "A OR B". (For instance, if you restart Exercise 9.1(a) and click on the A OR B sentence, the option to (circularly) apply the commutativity of or should be present. There should also be a "PROVED: OR IS COMMUTATIVE" line in your achievements window.

 **SansCipher** · 18 December 2018

I don't understand 22.1. What am I supposed to do there? I just don't understand how it works

>  **Terence Tao** · 18 December 2018
>
> Try dragging $Q(\alpha, \alpha)$ onto α (or vice versa). This will not give you exactly the required conclusion, but it should give a hint as to what one needs to do to $Q(\alpha, \alpha)$ first before you can get that conclusion. (Also it is worth reading all of the notes for this question carefully.)

 **lkimelfeld** · 28 December 2018

Thank you.
Interesting QED game:-)
Also it is easy and interesting to code exercises from it for the Leanprover (https://leanprover.github.io/)
I've coded several just for fun:
https://github.com/lkimelfeld/qedlean/blob/master/qed-lean.lean

 **Duncan Whyte** · 12 April 2019

In 9.2(b), the shortest proof with 12 lines uses exercise 3.1(a). I can do 3.1(a) in two moves (B -> B or C -> A or (B or C)) but not in one line. This means that the best I can get is 14 lines.
Which variables/formulas should be dragged onto which to get 3.1(a)?
Thanks in advance!

>  **Terence Tao** · 12 April 2019
>
> To select three objects, click on object 1, CTRL-click on object 2, and then CTRL-click on object 3. (I've now added some text to 3.1(a) to mention this; previously it was only mentioned in 9.4(c).)

 **Contador De HistóriasParaAdultos** · 25 May 2021

Great work



Anonymous · 26 October 2023

Hello , Where is the solutions ?



Anonymous · 9 October 2026

In Exercise 2.2(b), it says that right associativity is provable in 7 lines. That is possible, but I'm confused as the game seems to include the "Q.E.D." in the lines, meaning the actual proof is 6 lines. Am I missing something?